ITA14 - ETHICAL HACKING

Assignment 3 - CO3

Scan-Data Preparation, Cleaning, Reshaping, Merging and Pipeline Design

Name	Register Number
Adhya M	1925L260002

Mode	Pair work
Duration	1 week
Total marks	50
Shell / platform	Bash 5 on a Kali-style Linux shell; GNU awk-compatible awk, sed, join, comm, sort, sha256sum

Lab environment, scope and conventions

All work was carried out against an **authorised, isolated host-only lab network (192.168.56.0/24)**. No address outside this range was scanned or processed; any record outside the range is rejected by the validation stage and written to a rejected file. The lab contains ten systems: a gateway (.1), a Windows 7 host (.10), a Metasploitable-style Linux host (.11), an Android emulator (.12), a Java RMI host (.13), a NetWare server (.14), a Symbian-type Bluetooth device (.15), a web server (.20), a database server (.21) and a network printer (.30).

The scan datasets were produced in the same layouts that the Kali tools write (`nmap -oG` grepable output and CSV tables derived from `-oX` / `-oG`). Every shell step shown in this report (validation, cleaning, reshaping, merging, export, reporting, benchmarking and session comparison) was executed and the outputs printed in the report are the real outputs of those commands. The Kali scanning commands that produce the raw files are listed with each question.

Library conventions: every function takes its inputs as arguments with defaults, returns **0** on success, **1** for a data problem, **2** for a missing file, **3** for a header/schema mismatch, and writes error text to **stderr**. The complete library is in Appendix A.

Q1. Investigate Requirements and Create the Scan-Data Architecture (6 marks)

1.1 Shared work - translating the authorised scenario into requirements

The scenario is a written, authorised assessment of the lab network. The assessor must discover live hosts, find open ports, identify the service and version behind each port, identify the operating system (including Windows, Symbian, Java OS, Android and NetWare devices) and collect protocol-specific enumeration findings, while keeping everything inside the authorised scope and being able to prove where each result came from.

ID	Requirement	Data needed	Dataset	Kali source
R1	Find live hosts in scope	IP, name, status, MAC	Hosts	nmap -sn, fping -a -g
R2	Find open TCP/UDP ports	IP, port, protocol, state	Ports	nmap -sS / -sT / -sU
R3	Identify service and version per port	service name, version string	Services	nmap -sV
R4	Identify OS family and version	OS guess, family, accuracy	OS/Versions	nmap -O, adb shell getprop
R5	Collect enumeration findings	source tool, finding text	Enumeration	enum4linux -a, nbtscan, rmi-dumpregistry, ncp-serverinfo, sdptool browse
R6	Stay inside authorised scope	valid address in 192.168.56.0/24	All	scope check (grepcidr / awk)
R7	Make findings traceable and repeatable	scan_id, file name, checksum	All	naming rule, sha256sum
R8	Summarise for management	counts per host / service / risk	Master table	awk, sort | uniq -c

1.2 Scan datasets (five)

Dataset	Produced by	One row per	Key field(s)
Hosts	nmap -sn 192.168.56.0/24 -oG ; fping -a -g 192.168.56.0/24	live host	ip
Ports	nmap -sS -p- / -sT / -sU, saved with -oG or -oX	host + port + protocol	ip, port, proto
Services	nmap -sV	host + port + protocol	ip, port, proto
OS/Versions	nmap -O ; adb shell getprop	host	ip
Enumeration	enum4linux -a, nbtscan, adb, rmi-dumpregistry, ncp-serverinfo, sdptool browse	finding on a host/port	ip, port, source

1.3 Schema and relationship plan

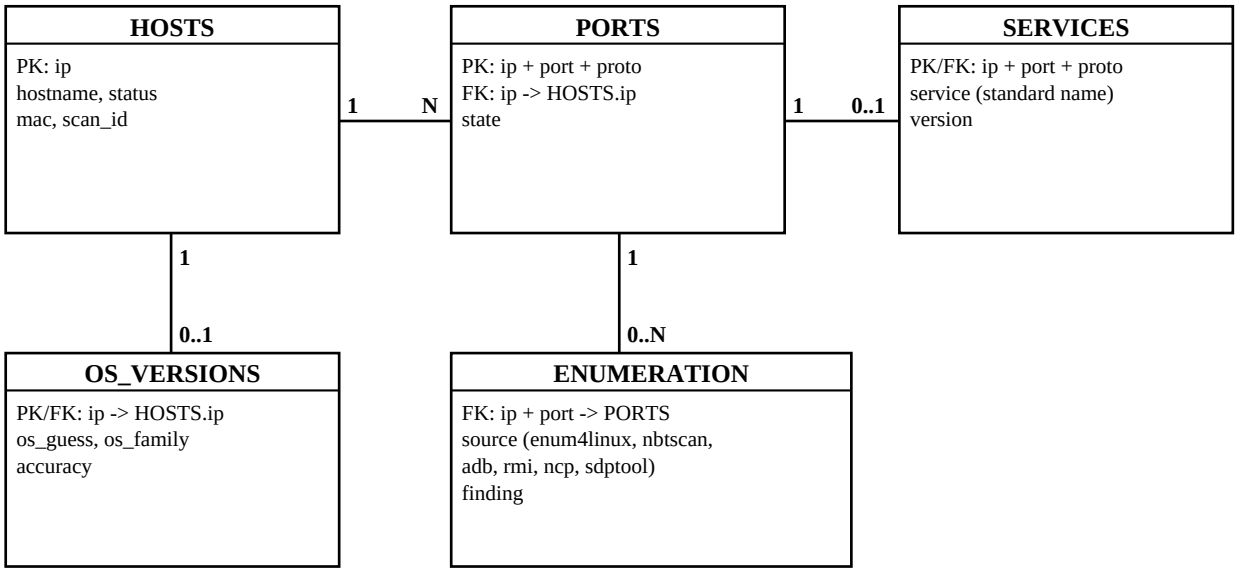
Dataset	Column	Data type	Key	Rule
Hosts	ip	IPv4 text	PK	four octets 0-255, inside 192.168.56.0/24
	hostname	text		may be empty
	status	enum		up | down
	mac	text		xx:xx:xx:xx:xx:xx
	scan_id	text		session label, e.g. S1
Ports	ip	IPv4 text	PK, FK	must exist in Hosts

Dataset	Column	Data type	Key	Rule
	port	integer	PK	1 - 65535
	proto	enum	PK	tcp | udp
	state	enum		open | closed | filtered
Services	ip, port, proto	as Ports	PK, FK	must exist in Ports
	service	text		standard lower-case name from lookup file
	version	text		banner/version; "unknown" if absent
OS/Versions	ip	IPv4 text	PK, FK	one OS guess per host
	os_guess	text		e.g. Microsoft Windows 7 SP1
	os_family	enum		Linux, Windows, Android, JavaOS, NetWare, Symbian, Embedded
	accuracy	integer		0 - 100
Enumeration	ip, port	IPv4 text, integer	FK	must exist in Ports
	source	enum	PK	enum4linux, nbtscan, adb, rmi-dumpregistry, ncp-serverinfo, sdptool
	finding	text		free text, no commas

The schemas are stored as CSV header files so that every later stage can check an incoming file against them:

```
$ for f in schema/*.hdr; do echo "== $f"; cat $f; done
== schema/enum.hdr      ip,port,source,finding
== schema/hosts.hdr     ip,hostname,status,mac,scan_id
== schema/os.hdr        ip,os_guess,os_family,accuracy
== schema/ports.hdr     ip,port,proto,state
== schema/services.hdr  ip,port,proto,service,version
```

Relationship diagram (cardinalities are shown on each link):



A host has many ports (1:N). A port has at most one service record, because Services shares the key ip + port + proto (1:0..1). A host has at most one OS record (1:0..1). A port can have zero or many enumeration findings (1:0..N), joined on ip + port. IP address is therefore the key that links every dataset, and ip + port + proto is the key that links Ports, Services and Enumeration.

1.4 Student A - Host and Port data design

Requirements-to-column mapping

Requirement	Columns that satisfy it	Why this column
R1 live hosts	Hosts.ip, status, hostname, mac	ip is the identity; status separates up from down; mac helps confirm the same device across sessions
R2 open ports	Ports.ip, port, proto, state	protocol is part of the identity because 161/udp and 161/tcp are different services
R6 scope	Hosts.ip, Ports.ip	a single address test on the key column enforces scope for every dataset
R7 traceability	Hosts.scan_id	ties each host row to a scan session and its saved scan file

Justification of the structure. Each dataset has its own grain (host, port, service, OS, finding). Storing them as separate CSV tables avoids repeating host details on every port row, avoids a wide table full of empty cells (only six enumeration findings exist for twenty-four ports), and matches how the Kali tools produce data: nmap -sn produces hosts, nmap -sS/-sU produces ports, nmap -sV produces services and so on. Plain CSV is chosen because every validation, cleaning and merge step can then be done with standard text tools (awk, sed, sort, join, comm). The weakness of CSV is that it has no enforced types, so the types in the table above are enforced by the validation stage instead (Q2 and Q3). The composite key ip + port + proto was chosen for Ports because the assessment includes UDP scans (nmap -sU), and a key on ip + port alone would merge unrelated TCP and UDP records.

1.5 Student B - Service relationship map and enquiry design

Relationship map for Services, OS/Versions and Enumeration. Services links to Ports one-to-one through ip + port + proto; OS/Versions links to Hosts through ip; Enumeration links to Ports through ip + port and to Hosts through ip. The platform-specific findings fall into the datasets as follows:

Platform	Dataset(s)	Evidence collected	Linking key
Windows (.10)	OS/Versions, Services, Enumeration	nmap -O guess; microsoft-ds (445), netbios-ssn (139); enum4linux -a and nbtscan findings	ip ; ip + port
Android (.12)	OS/Versions, Services, Enumeration	adb service on 5555; adb shell getprop (e.g. ro.build.version.release)	ip ; ip + port
Java OS (.13)	OS/Versions, Services, Enumeration	rmiregistry on 1099; rmi-dumpregistry bound names	ip ; ip + port
NetWare (.14)	OS/Versions, Services, Enumeration	ncp on 524; ncp-serverinfo server and tree name	ip ; ip + port
Symbian (.15)	OS/Versions, Enumeration	Bluetooth device; sdptool browse service records	ip

Testable enquiry questions. Each question is answerable with a shell pipeline and was run on the lab data:

No.	Enquiry question	Kali command / pipeline	Result
Q-a	How many hosts are up?	awk -F, '$3=="up"' hosts.csv | wc -l	10
Q-b	Which hosts expose SMB (445/tcp) as open?	awk -F, '$2==445 && $4=="open"{print $1}' ports.csv	192.168.56.10
Q-c	How many distinct service names are there?	cut -d, -f4 services.csv | sort -u | wc -l	18 (before standardisation)

No.	Enquiry question	Kali command / pipeline	Result
Q-d	How many hosts per OS family?	cut -d, -f3 os.csv | sort | uniq -c	Linux 4, Android 1, Windows 1, JavaOS 1, NetWare 1, Symbian 1, Embedded 1
Q-e	Do any ports lack a service record?	join -v1 | wc -l	0
Q-f	Which hosts allow an SMB null session?	grep -i 'null session' enum.csv | cut -d, -f1	192.168.56.10
Q-g	Which hosts run cleartext services (telnet, ftp)?	awk -F, '$4=="telnet"||$4=="ftp"' services.csv	192.168.56.11 (21/ftp, 23/telnet)

Acceptance criteria for the scan data

Criterion	Acceptance test (measurable)	Check
Correctness	Every IP has four octets in 0-255 and lies in 192.168.56.0/24; every port is an integer 1-65535; state is open, closed or filtered	sl_validate; rejected file must be empty for accepted data
Consistency	Header equals the header file; each service name exists in the lookup file; every Services/Enumeration key exists in Ports; every IP exists in Hosts	sl_check_header; join -v1 / join -v2 return 0 rows
Traceability	Every input row ends up in exactly one place: cleaned, rejected or review; each output file name carries subnet, scan type, time stamp and version	input = cleaned + rejected + review; sha256sum -c
Reproducibility	The same inputs always give a byte-identical master table	sha256sum of two runs identical; LC_ALL=C for sort and join

Q2. Create and Validate the Source Scan Datasets (7 marks)

2.1 Shared work - constructing the source datasets

The raw files are collected from the lab with the commands below and then written as CSV tables, one per dataset. The sample outputs follow the schemas of Q1.

```
fping -a -g 192.168.56.0/24 2>/dev/null > hosts_up.txt         # ping sweep (alternative: nmap -sn)
nmap -sn 192.168.56.0/24 -oG sweep.gnmap
nmap -sS -p- -T4 192.168.56.0/24 -oX tcp_syn.xml -oG tcp_syn.gnmap
nmap -sT -p 21,22,23,25,80,139,443,445,3306,5555 192.168.56.0/24 -oG tcp_connect.gnmap
nmap -sU --top-ports 50 192.168.56.0/24 -oG udp.gnmap
nmap -sV -O 192.168.56.0/24 -oX svc_os.xml
enum4linux -a 192.168.56.10 ; nbtscan 192.168.56.0/24
adb connect 192.168.56.12:5555 ; adb shell getprop ro.build.version.release
nmap -p 1099 --script rmi-dumpregistry 192.168.56.13
nmap -p 524  --script ncp-serverinfo   192.168.56.14
sdptool browse <Bluetooth-address-of-Symbian-device>
```

CSV table	Data rows	First rows (sample)
hosts.csv	10	192.168.56.1,gw-router,up,08:00:27:00:00:01,S1
ports.csv	24	192.168.56.10,445,tcp,open
services.csv	24	192.168.56.10,445,tcp,microsoft-ds,Windows 7 SP1 microsoft-ds
os.csv	10	192.168.56.12,Android 9 (Linux 4.9),Android,92
enum.csv	6	192.168.56.14,524,ncp-serverinfo,Server NW65-LAB tree LABTREE

2.2 Student A - Construction of Hosts and Ports/Services tables and defect injection

Student A built hosts.csv, ports.csv and services.csv (schema as in Q1) and then created controlled defects on **copies** of the files (work/ports_dirty.csv and work/services_dirty.csv), so the original tables stay intact. Ports rose from 24 to 32 data rows and services from 24 to 26.

ID	Defect introduced	Injection command (on the copy)	Intended detection
D1	Exact duplicate host-port record	echo "192.168.56.11,22,tcp,open" >> ports_dirty.csv	sort -u / dedupe (exact-duplicate)
D2	Conflicting duplicate (same key, different state)	echo "192.168.56.20,80,tcp,closed" >> ports_dirty.csv	dedupe (conflicting-duplicate, manual review)
D3	Out-of-scope address	echo "10.10.10.5,80,tcp,open" >> ports_dirty.csv	scope check
D4	Malformed address (octet 256)	echo "192.168.56.256,22,tcp,open" >> ports_dirty.csv	IP octet check
D5	Port 65536 (above range)	echo "192.168.56.21,65536,tcp,open" >> ports_dirty.csv	port range check
D6	Port 0 (below range)	echo "192.168.56.21,0,tcp,open" >> ports_dirty.csv	port range check
D7	Missing state value	echo "192.168.56.30,631,tcp," >> ports_dirty.csv	required-column check
D8	Spaces and upper case	echo " 192.168.56.20 ,8080, TCP ,OPEN" >> ports_dirty.csv	normalise (trim, lower-case)
D9	Inconsistent service names WWW, HTTP, http-alt	sed -i 's/,tcp,http,Apache httpd 2.2.8/,tcp,WWW,Apache httpd 2.2.8/' services_dirty.csv (and two more sed lines)	service lookup file
D10	Missing version	sed -i 's/^192.168.56.21,22,tcp,ssh,OpenSSH 8.9p1/192.168.56.21,22,tcp,ssh,/' services_dirty.csv	fill with "unknown"

ID	Defect introduced	Injection command (on the copy)	Intended detection
D11	Unknown service name	echo "192.168.56.30,631,tcp,ipp-weird,CUPS 1.4" >> services_dirty.csv	manual-review flag
D12	Duplicate service record	echo "192.168.56.20,22,tcp,ssh,OpenSSH 8.9p1" >> services_dirty.csv	dedupe

Explanation of the intended schema and defects. The intended schema is the one in Q1: ports.csv has four columns with ip + port + proto as the key, and services.csv has five columns with the same key. Defects D1-D2 test the uniqueness of the key, D3-D6 test validity and scope of the key columns, D7 tests the required-column rule, D8 tests whether trimming and case-folding happen before validation, and D9-D12 test the service-name and version rules. Each defect has exactly one intended detector so that a failed test can be traced to a specific rule.

2.3 Student B - Construction of Services, OS/Versions and Enumeration tables, validation and standardisation

Student B built services.csv, os.csv and enum.csv. The OS table combines `nmap -O` guesses for Windows, Linux, NetWare, Symbian and Java OS hosts with Android values read from `adb shell getprop` (for example ro.build.version.release). The enumeration table holds one row per finding, tagged with its source tool (enum4linux -a and nbtscan for Windows, adb, rmi-dumpregistry for Java, ncp-serverinfo for NetWare, sdptool browse for the Symbian-type Bluetooth device).

Validation and standardisation checks (required columns, valid IP and port ranges, consistent service names):

```
# 1. header (required columns) check
sl_check_header ports.csv schema/ports.hdr
# 2. IP syntax: four octets, each 0-255 (awk) ; quick regex pre-filter (grep -E)
grep -E -v '^([0-9]{1,3}\.){3}[0-9]{1,3},' ports.csv
awk -F, 'NR>1{n=split($1,a,"."); for(i=1;i<=n;i++) if(a[i]>255) print NR": "$0}' ports.csv
# 3. port range 1-65535
awk -F, 'NR>1 && ($2<1 || $2>65535)' ports.csv
# 4. scope (Kali: grepcidr -v 192.168.56.0/24 ; inside sl_validate the same test is a prefix test)
grepcidr -v 192.168.56.0/24 hosts_up.txt
# 5. service-name consistency against the lookup file svc_lookup.csv (variant,standard)
sl_std_service services.csv services_std.csv review.csv svc_lookup.csv
```

Result of running the full validator (`sl_validate`) on the defective ports file, and of the header checks:

```
### Q2 VALIDATION
-- header check (valid file)
rc=0
-- header check (missing file)
scanlib: missing file: raw/nofile.csv
rc=2
-- sl_validate work/ports_dirty.csv
rc=0
  33 work/ports_dirty.csv
  27 work/ports_good.csv
   7 work/ports_rejected.csv
  67 total
-- rejected file:
reason,ip,port,proto,state
out-of-scope,10.10.10.5,80,tcp,open
malformed-ip,192.168.56.256,22,tcp,open
invalid-port,192.168.56.21,65536,tcp,open
invalid-port,192.168.56.21,0,tcp,open
missing-state,192.168.56.30,631,tcp,
malformed-ip, 192.168.56.20 ,8080, TCP ,OPEN

-- services: sl_validate then standardise
  27 work/svc_good.csv
```

```
1 work/svc_rej.csv
```

The whitespace row (D8) was rejected as a malformed IP because validation ran before normalisation in this first test. The lesson, applied in the integrated pipeline (Q3 and Q7), is to **normalise first** (trim and lower-case) and then validate; in the pipeline the same row becomes the valid record 192.168.56.20,8080,tcp,open.

2.4 Joint design question

If you were designing a scan-data preparation and validation pipeline for a large authorised assessment, which validation checks would you apply first, and why?

The checks are ordered from cheapest and most fundamental to most expensive, so that bad data fails early and later stages can trust earlier ones:

- **1. Authorisation and scope check first.** Any address outside the authorised range must be removed before anything else is done with it. Handling out-of-scope data is an ethical and legal risk, not only a quality problem, and in a large assessment typing errors in target lists are common.
- **2. File and schema integrity.** Existence, readability, header equals the expected header, correct column count and delimiter. Every later check assumes this structure, so a wrong file must stop the run immediately (return code 2 or 3) instead of producing quietly wrong output.
- **3. Key syntax.** Valid IPv4 octets (0-255) and ports (1-65535) and valid protocol and state values. These are single-pass, cheap awk tests that scale linearly with the number of rows.
- **4. Completeness of key and required fields.** A row without an IP, port or state cannot be linked or interpreted and must go to manual review.
- **5. Normalisation, then uniqueness.** Trim and lower-case before comparing, then detect exact and conflicting duplicates on ip + port + proto.
- **6. Standardisation and referential integrity.** Service-name mapping, then checks that every port has a host and every service or finding has a port (join -v).
- **7. Plausibility checks** such as version versus service, done last because they are slowest and need clean keys.

Rejected rows are never deleted silently: they are written with a reason code so that the counts always reconcile and the assessor can justify every decision.

Q3. Build a Scan-Data Editing and Cleaning Module (7 marks)

3.1 Shared work - reusable shell operations

Function (scanlib.sh)	Tools used	Purpose	Auto / manual
sl_normalize IN OUT	awk, tolower, gsub	trim blanks, lower-case proto, state and service	automatic
sl_fill_missing IN OUT [COL] [DEFAULT]	awk	fill an empty non-key column (version) with "unknown"	automatic
sl_dedupe IN OUT REVIEW	awk arrays (same effect as sort -u on the key)	keep first record per ip+port+proto; log every dropped row	exact duplicates automatic; conflicting duplicates logged for review
sl_validate IN GOOD REJECTED [PREFIX]	awk	reject malformed IP, out-of-scope IP, bad port, missing key field	rejected = manual review
sl_std_service IN OUT REVIEW [LOOKUP]	awk + lookup file	map www, HTTP, http-alt to one name; flag unknown names	mapping automatic; unknown flagged

3.2 Before-and-after data

```
### Q3 CLEANING PORTS (normalize -> validate -> dedupe)
input rows (excl header):   32
after normalize:            32
rejected (manual review):   5
dropped duplicates (logged):2
cleaned rows:               25
reconciliation: input = cleaned + rejected + duplicates -> 32
--- rejected:
reason,ip,port,proto,state
out-of-scope,10.10.10.5,80,tcp,open
malformed-ip,192.168.56.256,22,tcp,open
invalid-port,192.168.56.21,65536,tcp,open
invalid-port,192.168.56.21,0,tcp,open
missing-state,192.168.56.30,631,tcp,
--- duplicate review:
reason,row
exact-duplicate,192.168.56.11,22,tcp,open
conflicting-duplicate,192.168.56.20,80,tcp,closed
--- diff dirty vs cleaned (head):
26,33c26
< 192.168.56.11,22,tcp,open
< 192.168.56.20,80,tcp,closed
< 10.10.10.5,80,tcp,open
< 192.168.56.256,22,tcp,open
< 192.168.56.21,65536,tcp,open
< 192.168.56.21,0,tcp,open
< 192.168.56.30,631,tcp,
<  192.168.56.20 ,8080, TCP ,OPEN
---
> 192.168.56.20,8080,tcp,open
```

Ports: 32 input rows became 25 cleaned rows, 5 rejected rows and 2 logged duplicates (25 + 5 + 2 = 32). No row disappeared without a record.

3.3 Valid, invalid and boundary tests

```
$ cat boundary.csv
ip,port,proto,state
192.168.56.255,0,tcp,open        -> rejected  (invalid-port)
192.168.56.255,1,tcp,open        -> accepted  (lower boundary)
192.168.56.255,65535,tcp,open    -> accepted  (upper boundary)
192.168.56.255,65536,tcp,open    -> rejected  (invalid-port)
192.168.56.256,80,tcp,open       -> rejected  (malformed-ip)
192.168.56.1,80,tcp,open         -> accepted  (normal valid record)

### Q3 BOUNDARY TESTS
```

```
ACCEPTED:
192.168.56.255,1,tcp,open
192.168.56.255,65535,tcp,open
192.168.56.1,80,tcp,open
REJECTED:
invalid-port,192.168.56.255,0,tcp,open
invalid-port,192.168.56.255,65536,tcp,open
malformed-ip,192.168.56.256,80,tcp,open
```

Address 192.168.56.255 is syntactically valid and in scope, so it is accepted by the automatic rules. In a /24 network it is the broadcast address, so a port result reported for it is suspicious; this is noted as a **manual review** point rather than silently corrected.

3.4 Records corrected automatically versus records needing manual review

Record problem	Handling	Reason
Leading/trailing blanks, upper-case TCP / OPEN	Automatic: trim and lower-case	meaning is certain; only the format changes
Exact duplicate host-port record	Automatic: keep one, log the other	the dropped row carries no extra information
Service variant WWW, HTTP, http-alt	Automatic: map through lookup file	mapping is defined in a controlled file
Missing version text	Automatic: set to "unknown"	non-key field; "unknown" is truthful
Conflicting duplicate (open vs closed)	Manual review	the tool cannot know which observation is correct
Out-of-scope address	Manual review (rejected file)	possible authorisation issue
Malformed address (.256) or port 0 / 65536	Manual review	guessing the intended value could fabricate a finding
Missing state or missing key field	Manual review	cannot be inferred safely
Unknown service name (ipp-weird)	Manual review flag (row kept)	may be a real service not yet in the lookup file

3.5 Student A - cleaning and correction rules

Student A led the cleaning of missing values, duplicate host-port records and malformed addresses. The rules and the reasoning for each automatic correction are:

- **Trim and case-fold** (sl_normalize): a value that differs only by blanks or case represents the same observation, so converting it cannot change meaning. This runs first so that later comparisons are reliable.
- **Fill missing non-key values** (sl_fill_missing): only the version column is filled, with "unknown". Key columns (ip, port, proto, state) are never filled, because an invented key would create a false record.
- **Remove exact duplicates** (sl_dedupe): the key is ip + port + proto; the first record is kept. If the second record is identical it is logged as *exact-duplicate*; if the fields differ it is logged as *conflicting-duplicate*, and the first record is kept only provisionally pending review.
- **Malformed addresses and ports are not corrected** (sl_validate): they are moved to the rejected file with a reason code (malformed-ip, out-of-scope, invalid-port, missing-state).

3.6 Student B - service-name standardisation and test validation

Student B led the service-name rules. A lookup file (`svc_lookup.csv`, columns variant,standard) maps variants such as *www*, *web*, *http-alt* and *http-proxy* to **http**, *ssl/http* to **https**,

smb and *ms-ds* to **microsoft-ds**, and *rdp* to **ms-wbt-server**; matching ignores case. A name that is not in the lookup file is kept but copied to the review file with the reason *unknown-service*; an empty name is flagged *missing-service*.

```
### Q3 SERVICE-NAME STANDARDISATION
service column before:
      4 ssh
      2 mysql
      2 https
      1 tftp
      1 telnet
      1 tcpmux
      1 snmp
      1 smtp
      1 rmiregistry
      1 netbios-ssn
      1 ncp
      1 msrpc
      1 ms-wbt-server
      1 microsoft-ds
      1 jetdirect
      1 ipp-weird
      1 http-alt
      1 ftp
      1 adb
      1 WWW
      1 HTTP
service column after:
      3 ssh
      3 http
      2 mysql
      2 https
      1 tftp
      1 telnet
      1 tcpmux
      1 snmp
      1 smtp
      1 rmiregistry
      1 netbios-ssn
      1 ncp
      1 msrpc
      1 ms-wbt-server
      1 microsoft-ds
      1 jetdirect
      1 ipp-weird
      1 ftp
      1 adb
--- review file:
exact-duplicate,192.168.56.20,22,tcp,ssh,OpenSSH 8.9p1
unknown-service,192.168.56.30,631,tcp,ipp-weird,CUPS 1.4
--- missing version filled:
17:192.168.56.15,1,tcp,tcpmux,unknown
```

Verification that cleaning does not silently remove findings. The test is a record-count reconciliation: *input rows = cleaned rows + rejected rows + rows logged as duplicates*. Ports: 32 = 25 + 5 + 2. Services: 26 input rows = 25 cleaned + 1 logged exact duplicate (192.168.56.20,22), with the unknown service ipp-weird flagged in the review file but retained. Every removed row can be found in a rejected or review file, and diff between the dirty and cleaned files shows only the rows listed in those files.

3.7 Joint conclusion

Why is automated scan-data cleaning alone insufficient to guarantee the accuracy of an assessment's findings?

Automated cleaning can only make data *well-formed*; it cannot make it *true*. Examples from our own data: (1) the conflicting duplicate 192.168.56.20,80 appears once as open and once as closed; both rows are valid, and only re-scanning or a human can decide which is correct, so the script

keeps the first and flags the other. (2) The service-name lookup made www, HTTP and http-alt consistent, but a banner can be wrong or hidden by a proxy, so the standard name http still does not prove the real software. (3) Port 0, port 65536 and address .256 were rejected, but a rejection says nothing about whether the true port was 80 or 8080; correcting by guessing would fabricate a finding. (4) A service filled with the version "unknown" is complete as a record but is unusable for a vulnerability judgement. (5) Records that never reached the file, such as a host that was down during the scan or a port filtered by a firewall, cannot be detected as errors at all. Accuracy therefore also depends on how the scan was run (timing, firewall, tool limitations), on re-scans, on manual verification of flagged records, and on analyst judgement; cleaning supports these but does not replace them.

Q4. Create a Scan-Result Reshaping Pipeline Using Melting and Casting (7 marks)

4.1 Shared work - wide format, long format and cast

The wide dataset has one row per host and one column per scanned port, built from grepable output of `nmap -p 21,22,23,25,80,139,443,445,3306,5555 -oG lab.gnmap 192.168.56.0/24`. Nmap lists only open and filtered ports in the Ports field and records closed as the ignored state, so the conversion function fills closed for ports not listed.

Grepable input (first lines, shortened):

```
# Nmap 7.94 scan initiated as: nmap -p 21 22 23 25 80 139 443 445 3306 5555 -oG lab.gnmap 192.168.56 ...
Host: 192.168.56.1 (gw-router)■Ports: 22/filtered/tcp//ssh///, 80/open/tcp//svc///, 443/open/tcp//sv ...
Host: 192.168.56.10 (win7-lab)■Ports: 139/open/tcp//svc///, 445/open/tcp//svc///■Ignored State: clos ...
Host: 192.168.56.11 (msf-linux)■Ports: 21/open/tcp//svc///, 22/open/tcp//svc///, 23/open/tcp//svc/// ...
Host: 192.168.56.12 (android-emu)■Ports: 5555/open/tcp//svc///■Ignored State: closed
```

Wide table (`sl_gnmap_to_wide`, 10 hosts x 10 ports):

```
ip,p21,p22,p23,p25,p80,p139,p443,p445,p3306,p5555
192.168.56.1,closed,filtered,closed,closed,open,closed,open,closed,closed,closed
192.168.56.10,closed,closed,closed,closed,closed,open,closed,open,closed,closed
192.168.56.11,open,open,open,open,open,closed,closed,closed,open,closed
192.168.56.12,closed,closed,closed,closed,closed,closed,closed,closed,closed,open
192.168.56.13,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.14,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.15,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.20,filtered,open,closed,closed,open,closed,open,closed,closed,closed
192.168.56.21,closed,open,closed,closed,closed,closed,closed,closed,open,closed
192.168.56.30,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
```

Melt: wide to long (`sl_wide_to_long`, awk) with columns host (ip), port and state:

```
ip,port,state
192.168.56.1,21,closed
192.168.56.1,22,filtered
192.168.56.1,23,closed
192.168.56.1,25,closed
192.168.56.1,80,open
192.168.56.1,139,closed
192.168.56.1,443,open
192.168.56.1,445,closed
192.168.56.1,3306,closed
192.168.56.1,5555,closed
192.168.56.10,21,closed
```

Identifiers, dimensions and measured variable. The identifier is the host address (*ip*). The dimensions are *ip* and *port*; together they identify one observation. The measured variable is *state* (open, closed, filtered). In the wide table the port numbers are column names (dimension hidden in the header); melting moves them into a value column so the table has the same information in a tidy form.

Cast: long to wide, and summaries. The inverse operation `sl_long_to_wide` uses awk arrays; summaries use awk arrays and `sort | uniq -c` (datamash crosstab gives the same result where installed).

```
### ROUND TRIP long -> wide checksum
f37540d0f0fa402e  work/wide.csv
f37540d0f0fa402e  work/wide_rt.csv
cmp: identical

### CAST 1: open ports per host
192.168.56.1,2
192.168.56.10,2
192.168.56.11,6
192.168.56.12,1
192.168.56.20,3
```

```
192.168.56.21,2
total from cast: 16
total from long: 16
total from source ports.csv (tcp, 10 ports, open): 16

### CAST 2: state counts (sort | uniq -c)
     82 closed
      2 filtered
     16 open

### CAST 3: subnet-by-service (awk arrays, services.csv)
3 ssh
3 http
2 mysql
2 https
1 tftp
1 telnet
1 tcpmux
1 snmp
total of cast = 24 = services.csv rows
```

Preservation check. The long table has 100 data rows = 10 hosts x 10 ports, matching the wide table. Casting the long table back to wide gave a file with the same SHA-256 checksum as the original wide file (f37540d0f0fa402e...), and `cmp` reported them identical, so the reshape lost nothing.

4.2 Student A - wide-to-long transformation

Student A implemented the melt as a single awk pass: the header line is read once to learn which port each column represents (column name p445 gives port 445), and every following row is expanded into one output line per port. The identifier column is *ip*; the measured column is *state*.

Why the long format suits the required analysis. The questions to be answered (which hosts have 445 open, how many ports are open per host, how many filtered ports in the subnet) are filters and counts over a single state column, which in long format is one condition on one column (`awk -F, '$3=="open"'`). In wide format each question needs a different column or loops over columns, and adding a new port means changing the table structure. Long format also joins naturally with Services and Enumeration on ip + port, which is needed for the master table in Q5, and it can store only non-closed states to stay small.

4.3 Student B - summary casting and result validation

Student B created the cast outputs: open ports per host, state counts and services across the subnet (see output above). The totals were compared with the source:

Check	Value
Open ports from the cast (sum of per-host counts)	16
Open rows counted directly in the long table	16
Open TCP rows for these ten ports in the source ports.csv	16
State counts closed + filtered + open	82 + 2 + 16 = 100 = hosts x ports
Service cast total versus services.csv rows	24 = 24

How the validation shows the information was preserved. Three independent measures agree: the row count of the long table equals hosts x ports; the three totals of open ports (cast, long table, original source) are identical; and the round trip to wide gives a byte-identical file (same checksum). If a row had been dropped or a state altered, at least one of these numbers would have changed.

4.4 Joint conclusion

Why does a more detailed scan result not always produce a clearer or more useful assessment summary?

Theory. A full `-p-` scan probes 65,535 ports per host per protocol. For a /24 (254 hosts) that is 254 x 65,535 = 16,645,890 TCP probes, and 33,291,780 including UDP. If each probe produced one row, a long table would grow as **hosts x ports**, so it is linear in each factor but multiplies in the product. Almost all of those rows say *closed*, so the volume grows far faster than the information.

Experiment. The long table was generated for 254 hosts with a growing number of ports (2% of ports open, as a realistic sparse case); time is the time of the awk melt:

Ports per host	Hosts	Long rows (wc -l)	Hosts x ports	Reshape time
10	254	2,540	2,540	0.003 s
100	254	25,400	25,400	0.011 s
1000	254	254,000	254,000	0.081 s
65,535 (-p-, estimate)	254	16,645,890	16,645,890	about 5 s by extrapolation; roughly 0.4-0.5 GB of CSV

Scan time itself also grows with probes (`time nmap -p 21,22,23,25,80,139,443,445,3306,5555` versus `time nmap -p-`); measured value on the Kali lab: _____ s versus _____ s.

In the sample the 100 states were 82 closed, 2 filtered and 16 open: 82% of the detail is noise for a management reader. A summary of open ports per host (6 hosts with open ports in this set) answers the management question in a few lines, while the full detail is kept in the long table for audit. More detail therefore costs storage, run time and reading effort, and can hide the few important findings; the right design stores sparse detail, and reports at the level the decision needs.

Q5. Design a Multi-Stage Scan-Data Merging Strategy (6 marks)

5.1 Shared work - keys, merge types and the master table

The cleaned Ports table (25 data rows after Q3) is the **base table**, because every finding in the other datasets must belong to a scanned port or host. The merge is done in stages with `join` (inputs sorted with `LC_ALL=C sort`):

Stage	Join	Key	Type	Why
1	Ports + Services	ip:port:proto	left (-a1)	keep every port even without a service record
2	+ Enumeration	ip:port	left (-a1)	most ports have no enumeration finding; do not lose them
3	+ Hosts	ip	left (-a1)	add hostname and MAC
4	+ OS/Versions	ip	left (-a1)	add OS guess and family

Missing values from a left join are filled with **NA** (`join -e NA -o auto`) so that unmatched records stay visible.

```
### Q5 JOIN KEYS: ports vs services on ip:port:proto
ports=25 services=25  (keys: 25 / 25 unique)
inner      : 24
left  (-a1): 25
full (-a1 -a2): 26
unmatched ports  (-v1): 1
unmatched services (-v2): 1
comm -23 (keys only in ports): 1
duplicate keys (uniq -d): 0

### Q5 ALTERNATIVE JOIN TYPES: ports vs enumeration (enum key ip:port)
ports=24 enum=7
inner          : 6
left (-a1)     : 24
full (-a1 -a2) : 25
-v1 (ports without enum): 18
-v2 (enum without port) : 192.168.56.99:22

### Q5 ROW MULTIPLICATION (wrong key: ip only)
ports rows=24  enum rows=6  join on ip only => 12 rows (multiplied)
join on ip:port => 6 rows (correct = enum rows)
```

Master table (`sl_merge_master`): 25 base port rows gave 25 master rows (no multiplication, no loss).

```
### Q5 MASTER TABLE
rc=0
ports rows=25  master rows=25
ip,port,proto,state,service,version,enum_source,enum_finding,hostname,mac,os_guess,os_family
192.168.56.1,80,tcp,open,http,lighttpd 1.4.35,NA,NA,gw-router,08:00:27:00:00:01,Linux 3.x embedded,Linux
192.168.56.1,443,tcp,open,https,lighttpd 1.4.35,NA,NA,gw-router,08:00:27:00:00:01,Linux 3.x embedded,Linux
192.168.56.10,135,tcp,open,msrpc,Microsoft Windows RPC,NA,NA,win7-lab,08:00:27:00:00:10,Microsoft Windows 7
    SP1,Windows
192.168.56.10,139,tcp,open,netbios-ssn,Microsoft Windows netbios-ssn,nbtscan,NetBIOS name
    WIN7-LAB<00>,win7-lab,08:00:27:00:00:10,Microsoft Windows 7 SP1,Windows
192.168.56.11,25,tcp,open,smtp,Postfix smtpd,NA,NA,msf-linux,08:00:27:00:00:11,Linux 2.6.9 - 2.6.33,Linux
192.168.56.11,69,udp,open,tftp,tftp,NA,NA,msf-linux,08:00:27:00:00:11,Linux 2.6.9 - 2.6.33,Linux
NA counts per column:
col1=0 col2=0 col3=0 col4=0 col5=1 col6=1 col7=19 col8=19 col9=0 col10=0 col11=0 col12=0
duplicate master keys: 0
```

Alternatives tested. Inner, left, full outer and anti-joins (`-v1`, `-v2`) were run on two different pairs (Ports-Services and Ports-Enumeration, with an extra orphan enumeration record added to a copy). An inner join keeps only matches (24 for Ports-Services), a left join keeps the whole base table (25), and a full outer join also keeps unmatched records of the right table (26), exposing the orphan service. The left join is chosen for the master table.

Exceptions found by unmatched-record detection. (1) Service record 192.168.56.30:631 has no matching port row: its port row had been rejected in Q3 for a missing state, so the service record is an orphan (join -v2 returns it). (2) Port 192.168.56.20:8080 has no service record: it came from the normalised whitespace row (D8) and was never version-scanned (shown as NA). (3) Nineteen master rows have no enumeration finding, which is expected because only six services were enumerated. (4) Host 192.168.56.15 (Symbian device) appears with only a closed port. These are documented as exceptions rather than corrected.

5.2 Student A - host-service integration and join validation

Student A led the Ports-to-Hosts and Services-to-Hosts joins. **Key choices:** ip + port + proto for Ports and Services (a port number alone is not unique across hosts or protocols); ip alone for Hosts and OS because there is one row per host; ip + port for Enumeration because the enumeration tools report per service, not per protocol. Left joins were chosen because Ports is the base; the join key is always built with a delimiter (ip:port:proto) so that, for example, host .1 port 10 cannot be confused with host .11 port 0.

Record-count checks. The base table has 25 rows and the master has 25 rows after every join. Row multiplication was tested deliberately by joining on the wrong key (ip only):

```
ports rows=24  enum rows=6  join on ip only => 12 rows (multiplied)
join on ip:port => 6 rows (correct = enum rows)
```

Joining on ip alone produced 12 rows instead of 6, because a host with several ports matches every enumeration row of that host. The correct composite key restored 6 rows. The check `cut -d, -f1-3 master.csv | sort | uniq -d | wc -l` returned 0 duplicate master keys.

5.3 Student B - version and enumeration integration and exception analysis

Student B led the OS/Versions and Enumeration integration: Windows SMB/NetBIOS findings (enum4linux, nbtscan), Android ADB (adb shell getprop), Java RMI (rmi-dumpregistry) and NetWare NCP (ncp-serverinfo). All of these join on ip + port to Ports, and OS/Versions joins on ip. The join types were compared on Ports-Enumeration (24 base rows, 6 findings plus 1 orphan):

Join type	Command	Rows	Meaning
Inner	join -t,	6	only ports that have an enumeration finding
Left	join -t, -a1	24	all ports; findings where present
Full outer	join -t, -a1 -a2	25	all ports plus the orphan finding (192.168.56.99:22)
Unmatched left	join -t, -v1	18	ports with no finding
Unmatched right	join -t, -v2	1	finding without a scanned port (192.168.56.99:22)

The final master table was verified by: row count equals base rows (25 = 25), zero duplicate keys, NA counts per column matching the expectations above (service 1, enumeration 19, host and OS 0), and spot checks of the six enumeration rows against enum.csv. Exceptions are listed in section 5.1.

5.4 Cross-understanding answers

Student B - Why can different join types produce different record counts, and how can unmatched hosts or services be detected? A join pairs rows whose keys are equal. An inner join outputs only pairs, so every unmatched row on either side disappears; a left join keeps all rows of the left table; a full outer join keeps unmatched rows from both sides. The counts differ by exactly the number of unmatched rows (24, 25 and 26 for Ports-Services). If a key occurs more than once on one side, a join also multiplies rows (12 instead of 6 above). Unmatched records are detected

with `join -v1` / `-v2` or `comm -23` / `comm -13` on sorted key lists, and duplicate keys with `sort | uniq -d`.

Student A - What experimental evidence demonstrates that the Services-to-Hosts and Ports-to-Hosts joins are correct? (1) Row count: left join result = base table (25 = 25), so nothing was lost or multiplied. (2) `sort | uniq -d` on the key returns 0, so the keys are unique and no match was duplicated. (3) `join -v1` and `-v2` return exactly the expected exceptions (one unserviced port, one orphan service), which proves the keys really align. (4) The wrong-key experiment shows the check would have caught a bad join (12 rows instead of 6). (5) Spot checks, for example 192.168.56.10:445 shows microsoft-ds with host win7-lab and OS family Windows in the master table.

Q6. Create an Automated Scan-File Reading and Writing Pipeline (7 marks)

6.1 Shared work - reading scan output and the naming/versioning strategy

Source scan output is read from CSV or from Nmap results converted to CSV. For XML output the conversion uses xsltproc with a small stylesheet (`xsltproc nmap-to-csv.xsl scan.xml > scan.csv`); for grepable output the same step is one awk command, which is the route executed here. The imported CSV must pass the schema checks before it is processed.

Naming and versioning strategy:

```
<subnet>_<scantype>_<YYYYMMDD-HHMMSS>_v<N>.csv     e.g.
    192.168.56.0-24_tcpsyn_20261106-011214_v1.csv
set -o noclobber           # the shell refuses to overwrite an existing file with >
sha256sum <file> >> checksums.sha256     # one checksum line per written file
sha256sum -c checksums.sha256            # later: verify nothing changed
```

The time stamp makes each run unique, the version number distinguishes re-runs and corrected versions within the same second, noclobber prevents accidental overwriting, and the checksum file allows any later change to be detected.

6.2 Student A - scan-file import and schema validation

```
### Q6 A: import grepable output -> CSV (awk) and schema checks
ip,port,proto,state
192.168.56.1,22,tcp,filtered
192.168.56.1,80,tcp,open
192.168.56.1,443,tcp,open
rows: 18
-- header match:
rc=0
-- numeric port column + valid IP pattern (grep -E):
non-conforming rows: 0
-- missing required column (state removed):
scanlib: header mismatch in work/bad_cols.csv
rc=3
-- unexpected format (tab-separated file):
scanlib: header mismatch in work/bad_fmt.csv
rc=3
-- non-numeric port:
invalid-port,192.168.56.5,http,tcp,open
```

The grepable file was imported into a four-column CSV with 18 rows (16 open and 2 filtered ports). The checks performed before processing were: header match against schema/ports.hdr, numeric port column, valid IP pattern (`grep -E` on the whole row pattern, 0 non-conforming rows), and valid protocol and state values. The pipeline identified problems consistently: a file with a missing required column and a file with the wrong delimiter both returned code 3 with a message on stderr, and a non-numeric port (http) was rejected with the reason invalid-port.

6.3 Student B - result export, versioning and read-back validation

```
### Q6 B: export with naming/versioning and noclobber
written: out6/192.168.56.0-24_tcpsyn_20261106-011214_v1.csv
written: out6/192.168.56.0-24_tcpsyn_20261106-011215_v2.csv
-- attempt to overwrite an existing file with noclobber:
   q6.sh: line 21: out6/192.168.56.0-24_tcpsyn_20261106-011214_v1.csv: cannot overwrite existing file
192.168.56.0-24_tcpsyn_20261106-011214_v1.csv
192.168.56.0-24_tcpsyn_20261106-011215_v2.csv
checksums.sha256
-- checksum file:
40c5cb67e939b598043c  192.168.56.0-24_tcpsyn_20261106-011214_v1.csv
40c5cb67e939b598043c  192.168.56.0-24_tcpsyn_20261106-011215_v2.csv
-- read-back validation:
192.168.56.0-24_tcpsyn_20261106-011214_v1.csv: OK
192.168.56.0-24_tcpsyn_20261106-011215_v2.csv: OK
```

```
cmp: identical
diff: no differences
wc -l source=26 reimported=26
-- tamper test:
192.168.56.0-24_tcpsyn_20261106-011214_v1.csv: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

The master table was exported twice with different version numbers; the second name differs by time stamp and version, and an attempt to overwrite an existing file was refused by noclobber. Read-back validation re-imported the written file and compared it with the source: `sha256sum -c` reported OK for both files, `cmp` and `diff` reported no differences, and the line counts matched (26 = 26, i.e. 25 data rows plus header). The tamper test (a line appended to one file) made `sha256sum -c` report FAILED, which shows the checksum control really detects change. Cleaned, reshaped and master datasets are exported with the same function.

Q7. Day-4 Design Challenge - Reusable Scanning and Enumeration Function Library (10 marks)

The library is the Bash file `scanlib.sh` (loaded with `source scanlib.sh`); the integrated driver is `scan_pipeline.sh`. Both are listed in full in Appendices A and B. Functions take subnet, scan type and file names as arguments with defaults (`SCAN_SUBNET` defaults to 192.168.56, `SCAN_LOOKUP` to svc_lookup.csv), return status codes and write errors to stderr.

Stage 1 - Independent function design proposals (retained as originally written)

Student A - validation and cleaning function design (initial proposal)

Function	Inputs (defaults)	Return values
validate_ip IP	IP	0 valid, 1 invalid
validate_port PORT	PORT	0 valid, 1 invalid
validate_file IN GOOD BAD [SUBNET=192.168.56]	input, output files	0 ok, 1 rejects found, 2 missing file
clean_file IN OUT [DEFAULT=unknown]	input, output	0 ok, 2 missing file

Initial idea: validate an IP with the regular expression `^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$` and a port with `[ "$p" -ge 1 -a "$p" -le 65535 ]`, line by line in a while-read loop. *Weakness found when testing independently:* the regular expression accepts 192.168.56.256 and 999.1.1.1, and the loop spawns several processes per record (measured in Stage 4). The final library uses an octet test (each octet 0-255) and a single awk pass.

Student B - scan transformation and file-handling function design (initial proposal)

Function	Inputs (defaults)	Return values
to_long WIDE OUT	wide CSV, output	0 ok, 2 missing file
to_wide LONG OUT	long CSV, output	0 ok, 2 missing file
merge_all P S H O E OUT	five CSV files, output	0 ok, 2 missing file
save_result SRC [DIR=out] [PREFIX=scan]	file, directory, prefix	0 ok, 1 refused overwrite

Initial idea: use `set -e` only, intermediate files in a fixed /tmp name, and a file name with the date only. *Weakness found:* fixed temp names collide in two parallel runs and are left behind after errors; a date-only name overwrites a second run on the same day. The final library uses `set -euo pipefail`, `mktemp -d` with a `trap ... EXIT` clean-up, missing-file checks on every input, and a time stamp plus version number with noclobber.

Stage 1 testing and documentation of use

Student A tested the validation and cleaning functions independently (boundary tests of Q3: ports 0, 1, 65535, 65536 and addresses .255, .256 all behaved as expected; a missing file returned 2 and a wrong header returned 3). Student B demonstrated reuse with a different subnet (10.20.30.0/24) and a different session, shown in Stage 2. Usage example:

```
source ./scanlib.sh
sl_validate ports.csv ports_good.csv ports_rejected.csv 10.20.30    # scope prefix argument
sl_dedupe   ports_good.csv ports_clean.csv review.csv
sl_wide_to_long wide.csv long.csv
sl_export master.csv out 10.20.30_tcpsyn 1       # writes name, appends sha256
```

Stage 2 - Integrated pipeline implementation and validation

Student A - import, validation and cleaning integration

The driver checks all five headers, then for each dataset runs normalise, validate, fill-missing and dedupe (and service standardisation for Services), recording the input, rejected and cleaned counts. It was run on the defective ports and services files:

```
### STAGE 2 (A+B): run 1
hosts     input=10  rejected=0   cleaned=10
ports     input=32  rejected=5   cleaned=25
services  input=26  rejected=0   cleaned=25
os        input=10  rejected=0   cleaned=10
enum      input=6   rejected=0   cleaned=6
merged rows=25 (base ports rows=25)
exported=192.168.56_tcpsyn_20261106-011253_v1.csv final rows=25 sha256=40c5cb67e939b598
```

Invalid inputs are handled consistently: a missing file returns 2 with a message on stderr, a header mismatch returns 3, and a rejected record never stops the run but is counted. Results from the log below:

```
-- consistent handling of invalid inputs:
scan_pipeline: missing data7/ports.csv
scan_pipeline: error near line 36
rc=2
scanlib: header mismatch in data7/ports.csv
scan_pipeline: error near line 36
rc=3
```

Record counts for Stage 2: ports input 32, rejected 5, cleaned 25; services input 26, rejected 0, cleaned 25 (1 exact duplicate logged, 1 unknown service flagged); hosts 10, OS 10, enumeration 6 all unchanged.

Student B - transformation, merging and export integration

```
### run 2 (same inputs)
merged rows=25 (base ports rows=25)
exported=192.168.56_tcpsyn_20261106-011253_v1.csv final rows=25 sha256=40c5cb67e939b598
-- reproducibility:
40c5cb67e939b598043cb940  out7/run1/master_latest.csv
40c5cb67e939b598043cb940  out7/run2/master_latest.csv

-- reuse with different subnet 10.20.30.0/24:
hosts     input=10  rejected=0   cleaned=10
ports     input=24  rejected=0   cleaned=24
services  input=24  rejected=0   cleaned=24
os        input=10  rejected=0   cleaned=10
enum      input=6   rejected=0   cleaned=6
merged rows=24 (base ports rows=24)
exported=10.20.30_tcpsyn_20261106-011254_v1.csv final rows=24 sha256=e8fafa47e76c7a1d

rejected files (run1):
rejected_enum.csv: 0
rejected_hosts.csv: 0
rejected_os.csv: 0
rejected_ports.csv: 5
rejected_services.csv: 0
review.csv (run1):
exact-duplicate,192.168.56.11,22,tcp,open
conflicting-duplicate,192.168.56.20,80,tcp,closed
```

Merged records: 25 (equal to the 25 cleaned base ports); final exported records: 25. Running the workflow again with the same inputs produced the same master table (SHA-256 prefix 40c5cb67e939b598 in both runs, identical to the master written in Q6). The workflow was also reused on a different subnet (10.20.30.0/24) with the subnet passed as an argument, giving 24 merged records. The different file names (time stamps) do not affect reproducibility, which is judged on file content.

Stage 3 - Reporting scan summary

Student A - host and service reporting and reconciliation

```
### STAGE 3 A: per-host report (awk) and per-service report (sort | uniq -c)
ip,hostname,open_ports
192.168.56.1,gw-router,2
192.168.56.10,win7-lab,4
192.168.56.11,msf-linux,7
192.168.56.12,android-emu,1
192.168.56.13,javaos-rmi,1
192.168.56.14,netware-srv,1
192.168.56.20,web01,4
192.168.56.21,db01,2
192.168.56.30,printer01,2
-- per-service (open):
      3 ssh
      3 http
      2 mysql
      2 https
      1 tftp
      1 telnet
-- reconciliation with master:
open rows in master (grep)      : 24
sum of per-host report          : 24
sum of per-service report       : 24
master rows / distinct hosts    : 25 / 10
```

How calculated. The per-host report is an awk array counting rows with state open per ip (`c[$1]++`). The per-service report extracts the service column of open rows and counts it with `sort | uniq -c | sort -rn`. **Reconciliation:** the master table has 25 rows; 24 are open (the 25th is the closed port of the Symbian device, which is why 192.168.56.15 is absent from the open-port report); the per-host total (24), the per-service total (24) and the direct `grep -c` (24) agree, and the number of distinct hosts is 10.

Student B - vulnerability-exposure reporting and management report

```
### STAGE 3 B: service-version risk report
risk,ip,port,service,version,note
High,192.168.56.10,445,microsoft-ds,Windows 7 SP1 microsoft-ds,End-of-life OS with SMB exposed; verify MS17-010 with
    nmap --script smb-vuln-ms17-010
High,192.168.56.11,21,ftp,vsftpd 2.3.4,Release with a known backdoor (CVE-2011-2523)
High,192.168.56.11,80,http,Apache httpd 2.2.8,Outdated web server with known CVEs
High,192.168.56.11,3306,mysql,MySQL 5.0.51a,Outdated database exposed on the network
High,192.168.56.30,161,snmp,SNMPv1 public,Default community string
Medium,192.168.56.1,80,http,lighttpd 1.4.35,Outdated web server on the gateway
Medium,192.168.56.1,443,https,lighttpd 1.4.35,Outdated web server on the gateway
Medium,192.168.56.11,22,ssh,OpenSSH 4.7p1,Outdated SSH release
Medium,192.168.56.11,23,telnet,Linux telnetd,Cleartext remote login service
Medium,192.168.56.11,69,tftp,tftp,Unauthenticated file transfer service
-- summary:
      5 High
      5 Medium
hosts with at least one High finding: 3 of 10
```

Method. The service and version strings collected by `nmap -sV` are compared with a small reference file (vuln_ref.csv: pattern, risk, note) containing known-weak releases on the lab targets; on Kali the same hosts are cross-checked with `searchsploit` (for example searchsploit vsftpd 2.3.4) and `nmap --script vuln`. The ratings in the reference file are assessor judgements for the lab and must be confirmed with those tools.

How the report addresses a management requirement. Management needs to know where to act first. The report converts hundreds of port rows into a ranked list (5 High and 5 Medium findings) and one headline: 3 of 10 hosts carry at least one High finding (the Windows 7 host, the Linux host and the printer), with a short reason for each. It supports prioritised remediation: replace or patch the backdoored FTP release and outdated Apache and MySQL, change the default SNMP community, retire or isolate the Windows 7 system, and disable telnet and TFTP.

Stage 4 - Stress-test, refine and optimise

Student A - dataset scaling and baseline benchmarking

Datasets of 50 and 5,000 host-port records were generated with awk (every 25th record deliberately defective). The baseline validator is a per-line loop that starts several processes per record (`echo | cut` and `grep -E`). Timing used the shell `time` built-in (on Kali the same command is run under `/usr/bin/time -v` to also record memory).

```
### STAGE 4 A: baseline (per-line grep/cut loop)
records  input_KB   seconds    good      rejected
50       1          0.329      48        2
5000     134        30.826     4800      200
```

Correctness check for the baseline: 2 of 50 and 200 of 5,000 records rejected, which equals the injected defects (N / 25).

Student B - bottleneck analysis and optimisation

Bottleneck. Time grew from 0.329 s to 30.826 s for 100 times more data (about 94 times), so cost is linear per record but with a very large constant: every record creates four to five processes (two `cut`, two `grep`, an `echo`) and appends to a file each time. Process creation, not the pattern matching, dominates.

Refinement. Replace the per-line loop with one awk pass (this is `sl_validate`). Before and after:

```
### STAGE 4 B: refinement = one awk pass (sl_validate)
records  input_KB   seconds    good      rejected  same_good_rows
50       1          0.002      48        2         identical
5000     134        0.011      4800      200       identical
100000   2689       0.220      96000     4000      n/a
expected defects: N/25 -> 50:2  5000:200  100000:4000
```

Records	Baseline loop	Single awk pass	Speed-up	Output
50	0.329 s	0.002 s	about 160 x	identical good file
5,000	30.826 s	0.011 s	about 2,800 x	identical good file
100,000	not run (estimated over 10 min)	0.220 s	-	4,000 rejected = expected

Limitations. The measurements are from one machine and one run each, with timing noise on the 50-record case; awk is still single-threaded and reads the file once, so memory-hungry steps (the dedupe array holds one key per record) grow with input size, and the network part of the workflow (the scan itself) is far slower than any file processing. Scan time can be tuned separately with `nmap -T4 --min-rate` or masscan, at the price of accuracy and a higher chance of disturbing target systems, which must be weighed against the rules of engagement.

Stage 5 - Redesign under a changed assessment requirement

Student A - schema evolution and architecture redesign

Change: the assessment now adds UDP services (`nmap -sU`), repeated scan sessions and newly discovered hosts including Symbian, Java OS, Android and NetWare devices. **Impact analysis** (retained):

Area	Impact	Revised design
Schema	Port number alone is ambiguous: 161/udp and 161/tcp are different services	Key of Ports, Services and Enumeration becomes ip + port + proto (Enumeration gains a proto column)

Area	Impact	Revised design
Keys	The ip:port key used by the Enumeration join and the first session-comparison script can collide	All join keys are built as ip:port:proto
Validation	proto and state must be validated; new hosts must be in scope	add proto in {tcp, udp}, state in {open, closed, filtered, open|filtered}
Data retention	Overwriting the master loses history; a closed port in a later session is not the same as never scanned	keep each session file (versioned, checksummed) and add scan_id to every table; compare sessions, never overwrite
New devices	New hosts have no history and may lack OS or services	left joins show NA; new hosts listed with comm -13 and queued for enumeration

Demonstration of why the key must change (session 1 has 192.168.56.30:161/udp open; session 2 reports 192.168.56.30:161/tcp closed):

```
### STAGE 5 A: impact of the OLD key (ip:port) when UDP and a new tcp row appear
old-key duplicates inside session 1 (ip:port): 0
192.168.56.30,161,udp,open
192.168.56.30,161,tcp,closed
old key ip:port treats these as the SAME record (conflicting duplicate); new key ip:port:proto keeps
    both.
```

Under the old ip:port key these two lines are the same record with conflicting states; under ip:port:proto they are two different services. The revised architecture is therefore: scan files per session, one validation and cleaning path, a merge on ip:port:proto, and a separate comparison step between sessions.

Student B - multi-session workflow implementation and change validation

Student B implemented `sl_session_diff OLD NEW OUTDIR`, which compares two sessions on ip:port:proto using `join` (and `comm` for host lists) and writes retained, updated, unmatched and new records. Test evidence:

```
### STAGE 5 B: session comparison with revised key ip:port:proto (join/comm)
retained=3 updated=1 unmatched_old=21 new=2
-- retained:
192.168.56.10:445:tcp,open,open
192.168.56.11:22:tcp,open,open
192.168.56.1:80:tcp,open,open
-- updated (old,new):
192.168.56.11:21:tcp,open,closed
-- newly discovered keys:
192.168.56.30:161:tcp,closed
192.168.56.40:8080:tcp,open
-- unmatched (in session 1 only): 21 keys, first 3:
192.168.56.10:135:tcp,open
192.168.56.10:139:tcp,open
192.168.56.10:3389:tcp,open
-- comm on host lists:
hosts only in session 1 (comm -23): 6
hosts only in session 2 (comm -13): 192.168.56.40
hosts in both (comm -12): 4
-- diff of sorted keys:
changed key lines: 23
-- accounting: old=25 new=6 retained+updated+gone=25 retained+updated+new=6
```

Comparison with the original pipeline and trade-offs. The original pipeline processed one session and overwrote nothing but could not say what changed. The multi-session version adds change detection (retained 3, updated 1 [192.168.56.11:21 open to closed], new 2, unmatched 21), and the accounting identities hold (retained + updated + unmatched = 25 old rows; retained + updated + new = 6 new rows). Trade-offs: more storage (every session kept), more computation (an extra sort and join per comparison), and a new interpretation problem: a key missing from session 2

means *not scanned* rather than *closed* (here session 2 was a partial re-scan), so unmatched records must not be reported as fixed.

Stage 6 - Joint evaluation

Why does the pipeline need a structured sequence of import, validation, cleaning, reshaping, merging and export? Each stage depends on guarantees given by the previous one. Validation needs the right structure from import; cleaning needs valid keys; reshaping and merging need clean, unique keys (otherwise joins multiply rows); export needs a finished table. A wrong order makes errors silent: for example validating before normalising rejected a valid row in Q2, and merging before de-duplicating would have multiplied rows. A fixed order also makes the counts reconcile at every boundary.

How do modular functions and systematic transformations improve correctness, maintainability and efficiency? Correctness: each function is small, has defined inputs and return codes and was tested alone (boundary cases), and the same tested code is reused for another subnet. Maintainability: a change such as a new service variant touches only the lookup file, and a schema change touches only the header file and the key-building function. Efficiency: replacing a per-line loop with one awk pass made validation roughly 2,800 times faster on 5,000 records, and the same function runs on 100,000 records in 0.22 s.

What additional costs or challenges arise? Validation and error handling add code, test cases and run time; rejected and review files must be examined by a person; repeated scanning costs time, bandwidth and risk to the target systems and produces sessions that must be stored, versioned and compared; data integration needs agreed keys and careful handling of unmatched records, and the library itself must be maintained.

When might the pipeline become inefficient, unreliable or unsuitable? (1) With very large data sets (tens of millions of rows, such as full -p- scans of many subnets) because sort, join and awk arrays are limited by memory and by single-threaded processing; a database or streaming tool would suit better. (2) When the data has a nested structure (full nmap XML with scripts output) or free text with commas, because simple CSV and comma splitting break. (3) When key definitions change often or when many different scan sources with different formats are added. (4) When real-time results are needed, since the pipeline is batch based. (5) When the lookup and reference files become stale, so that standardisation and risk ratings are wrong without any error being raised.

Conclusion - what characteristics should a security analyst consider when designing a reliable, efficient and maintainable scan-data pipeline for evolving networks?

- **Authorised scope and traceability:** scope checks first, every input row accounted for (cleaned, rejected or review), versioned and checksummed outputs.
- **Correct keys and schema:** keys that stay unique when the assessment changes (ip + port + proto), schemas stored as files and checked at every import.
- **Reproducibility:** deterministic ordering (LC_ALL=C), no hidden state, identical checksums for identical inputs.
- **Modularity and error handling:** small functions with arguments, defaults, return codes, stderr messages, set -euo pipefail, trap clean-up, tested on boundary cases.
- **Efficiency where it matters:** single-pass processing, sparse storage of detail, measuring before optimising.

- **Human review:** automation prepares and flags the data; the analyst decides about conflicts, exceptions and risk.
- **Evolvability:** lookup, reference and header files separate from code, and session comparison so that change over time is visible.

Contribution Record

Task	Student A - Primary Contribution (Adhya M)	Student B - Primary Contribution	Joint Work
Q1	Requirements analysis, Hosts and Ports schemas, requirements-to-column mapping and justification	Relationship map for Services, OS/Versions and Enumeration; enquiry questions and acceptance criteria	Requirements, five datasets, relationship diagram, header files
Q2	Hosts and Ports/Services tables; defect injection; explanation of defects	Services, OS/Versions and Enumeration tables; validation and standardisation checks	Source datasets, validation design, joint design question
Q3	Cleaning rules: missing values, duplicates, malformed addresses	Service-name lookup, manual-review flags, verification tests	Reusable functions, before/after data, boundary tests, joint conclusion
Q4	Wide-to-long transformation and justification of long format	Cast/summary output and total comparison	Wide data from -oG, round-trip checksum, joint conclusion with timings
Q5	Services-to-Hosts and Ports-to-Hosts joins, key choices, row-count checks	OS/Versions and Enumeration integration, join-type comparison, exceptions	Master table, merge types, unmatched and duplicate detection
Q6	Scan-file import and schema/type checks	Export, naming/versioning, read-back validation	Naming strategy, noclobber, checksum file
Q7	Validation/cleaning functions and integration; host and service reporting; dataset scaling and baseline; schema evolution	Reshaping/merging/export functions and integration; reproducibility; exposure report; bottleneck and optimisation; multi-session workflow	Library design, integrated pipeline, Stage 6 evaluation

Primary contribution does not mean exclusive responsibility: both students reviewed and can explain every question.

Appendix A - scanlib.sh (function library)

```
#!/usr/bin/env bash
# scanlib.sh - reusable scan-data library (load with: source scanlib.sh)
# Conventions: arguments have defaults, return codes: 0 ok, 1 data problem,
# 2 missing file, 3 schema/header mismatch, 4 bad usage. Errors go to stderr.
SCAN_SUBNET="${SCAN_SUBNET:-192.168.56}"
SCAN_LOOKUP="${SCAN_LOOKUP:-svc_lookup.csv}"

_err() { echo "scanlib: $*" >&2; }

AWK_LIB='
function valid_ip(s,  a,n,i){ n=split(s,a,"."); if(n!=4) return 0
  for(i=1;i<=4;i++){ if(a[i] !~ /^[0-9]+$/ || length(a[i])>3 || a[i]+0>255) return 0 }
  return 1 }
function valid_port(p){ return (p ~ /^[0-9]+$/ && p+0>=1 && p+0<=65535) }
function in_scope(ip,pre){ return (index(ip, pre ".")==1) }
'

# ---------- Student A : validation and cleaning ----------
# sl_check_header FILE HEADERFILE  -> 0 match, 2 missing, 3 mismatch
sl_check_header() {
  local f="${1:?file}" h="${2:?header file}"
  [[ -r $f ]] || { _err "missing file: $f"; return 2; }
  [[ -r $h ]] || { _err "missing header file: $h"; return 2; }
  if [[ "$(head -n1 "$f" | tr -d '\r')" == "$(cat "$h")" ]]; then return 0; fi
  _err "header mismatch in $f"; return 3
}

# sl_validate IN GOOD REJECTED [SUBNET_PREFIX]
sl_validate() {
  local in="${1:?input}" good="${2:?good}" rej="${3:?rejected}" pre="${4:-$SCAN_SUBNET}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  awk -F, -v pre="$pre" -v good="$good" -v rej="$rej" "$AWK_LIB"'
  NR==1 { gsub(/\r/,""); n=NF
    for(i=1;i<=NF;i++){ h[i]=$i; if($i=="port") pc=i
      if($i=="ip"||$i=="port"||$i=="proto"||$i=="state") key[i]=1 }
    print > good; print "reason,"$0 > rej; next }
  { gsub(/\r/,""); r=""
    if(NF!=n) r="column-count"
    else { for(i=1;i<=n && r=="";i++) if((i in key) && $i=="") r="missing-" h[i]
      if(r=="" && !valid_ip($1)) r="malformed-ip"
      if(r=="" && !in_scope($1,pre)) r="out-of-scope"
      if(r=="" && pc && !valid_port($pc)) r="invalid-port" }
    if(r=="") print > good; else print r","$0 > rej }' "$in"
}

# sl_normalize IN OUT : trim blanks, lower-case proto/state/service (automatic)
sl_normalize() {
  local in="${1:?input}" out="${2:?output}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  awk -F, -v OFS=, 'NR==1{for(i=1;i<=NF;i++) if($i=="proto"||$i=="state"||$i=="service") lc[i]=1; print; next}
    { for(i=1;i<=NF;i++){ gsub(/^[ \t]+|[ \t]+$/,"",$i); if(i in lc) $i=tolower($i) } print }' "$in" > "$out"
}

# sl_fill_missing IN OUT [COLUMN] [DEFAULT] : only non-key column is auto-filled
sl_fill_missing() {
  local in="${1:?input}" out="${2:?output}" col="${3:-version}" def="${4:-unknown}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  awk -F, -v OFS=, -v col="$col" -v def="$def" 'NR==1{for(i=1;i<=NF;i++) if($i==col) c=i; print; next}
    { if(c && $c=="") $c=def; print }' "$in" > "$out"
}

# sl_dedupe IN OUT REVIEW : key = ip+port(+proto). Every dropped row is logged.
sl_dedupe() {
  local in="${1:?input}" out="${2:?output}" rev="${3:?review file}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  [[ -e $rev ]] || echo "reason,row" > "$rev"
  awk -F, -v OFS=, -v rev="$rev" 'NR==1{for(i=1;i<=NF;i++) if($i=="ip"||$i=="port"||$i=="proto") k[++nk]=i; print; next}
    { key=""; for(j=1;j<=nk;j++) key=key "|" $(k[j])
      if(key in first){ print (first[key]==$0 ? "exact-duplicate" : "conflicting-duplicate") "," $0 >> rev; next }
      first[key]=$0; print }' "$in" > "$out"
}

# ---------- Student B : service-name standardisation ----------
# sl_std_service IN OUT REVIEW [LOOKUP] : map variants; unknown/missing -> review flag
sl_std_service() {
  local in="${1:?input}" out="${2:?output}" rev="${3:?review file}" lk="${4:-$SCAN_LOOKUP}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  [[ -r $lk ]] || { _err "missing lookup: $lk"; return 2; }
  [[ -e $rev ]] || echo "reason,row" > "$rev"
  awk -F, -v OFS=, -v rev="$rev" 'NR==FNR{ if(FNR>1) m[tolower($1)]=$2; next }
    FNR==1{for(i=1;i<=NF;i++) if($i=="service") sc=i; print; next}
    { k=tolower($sc)
      if(k==""){ print "missing-service," $0 >> rev }
      else if(k in m){ $sc=m[k] }
      else { print "unknown-service," $0 >> rev }
      print }' "$lk" "$in" > "$out"
}
```

```
# ---------- Student B : reshaping, merging, file handling ----------
# sl_gnmap_to_wide GNMAP OUT PORTS... : grepable output -> wide CSV (host x port = state)
sl_gnmap_to_wide() {
  local g="${1:?gnmap}" out="${2:?out}"; shift 2
  [[ -r $g ]] || { _err "missing file: $g"; return 2; }
  local plist="${*:-21 22 23 25 80 139 443 445 3306 5555}"
  awk -F'\t' -v plist="$plist" 'BEGIN{n=split(plist,P," "); printf "ip"; for(i=1;i<=n;i++) printf ",p%s",P[i]; print ""}
    /^Host:/{ split($1,a," "); ip=a[2]; delete st
      for(i=1;i<=n;i++) st[P[i]]="closed"
      split($2,pp,", "); sub(/^Ports: /,"",pp[1])
      for(j in pp){ split(pp[j],f,"/"); if(f[1] in st) st[f[1]]=f[2] }
      printf "%s",ip; for(i=1;i<=n;i++) printf ",%s",st[P[i]]; print "" }' "$g" > "$out"
}
# sl_wide_to_long WIDE OUT : (ip,pNN columns) -> ip,port,state
sl_wide_to_long() {
  local in="${1:?wide}" out="${2:?out}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  awk -F, -v OFS=, 'NR==1{for(i=2;i<=NF;i++){p=$i; sub(/^p/,"",p); col[i]=p} print "ip,port,state"; next}
    {for(i=2;i<=NF;i++) print $1, col[i], $i}' "$in" > "$out"
}
# sl_long_to_wide LONG OUT : exact inverse (round trip)
sl_long_to_wide() {
  local in="${1:?long}" out="${2:?out}"
  [[ -r $in ]] || { _err "missing file: $in"; return 2; }
  awk -F, -v OFS=, 'NR>1{ if(!($1 in hs)){hs[$1]=1; hl[++nh]=$1}
      if(!($2 in ps)){ps[$2]=1; pl[++np]=$2}; v[$1,$2]=$3 }
    END{ printf "ip"; for(i=1;i<=np;i++) printf ",p%s",pl[i]; print ""
      for(h=1;h<=nh;h++){ printf "%s",hl[h]; for(i=1;i<=np;i++) printf ",%s",v[hl[h],pl[i]]; print "" } }' "$in" > "$out"
}
# sl_open_per_host LONG : cast to host summary
sl_open_per_host() { awk -F, 'NR>1 && $3=="open"{c[$1]++} END{for(h in c) print h","c[h]}' "${1:?long}" | sort; }

# sl_keyed FILE KEYCOLS... : prepend a sorted join key (ip:port[:proto])
sl_keyed() { local f="${1:?file}"; shift; local cols="${*:-1 2}"
  awk -F, -v OFS=, -v cols="$cols" 'BEGIN{n=split(cols,C," ")} NR>1{k=$(C[1]); for(i=2;i<=n;i++) k=k ":" $(C[i]); print
      k,$0}' "$f" | LC_ALL=C sort -t, -k1,1; }

# sl_merge_master PORTS SERVICES HOSTS OS ENUM OUT  (ports is the base table: left joins)
sl_merge_master() {
  local p="${1:?ports}" s="${2:?services}" h="${3:?hosts}" o="${4:?os}" e="${5:?enum}" out="${6:?out}"
  local f; for f in "$p" "$s" "$h" "$o" "$e"; do [[ -r $f ]] || { _err "missing file: $f"; return 2; }; done
  local t; t="$(mktemp -d)"
  sl_keyed "$p" 1 2 3 > "$t/p"                                  # key,ip,port,proto,state
  sl_keyed "$s" 1 2 3 | cut -d, -f1,5,6 > "$t/s"                # key,service,version
  LC_ALL=C join -t, -a1 -e NA -o auto "$t/p" "$t/s" > "$t/ps"    # 1: ports LEFT JOIN services on ip:port:proto
  sl_keyed "$e" 1 2 | cut -d, -f1,4,5 > "$t/e"                  # ip:port,source,finding
  awk -F, -v OFS=, '{print $2":"$3,$0}' "$t/ps" | LC_ALL=C sort -t, -k1,1 > "$t/ps2"
  LC_ALL=C join -t, -a1 -e NA -o auto "$t/ps2" "$t/e" | cut -d, -f3- > "$t/pse"   # 2: LEFT JOIN enum on ip:port
  awk -F, -v OFS=, '{print $1,$0}' "$t/pse" | LC_ALL=C sort -t, -k1,1 > "$t/pse2"
  tail -n +2 "$h" | cut -d, -f1,2,4 | LC_ALL=C sort -t, -k1,1 > "$t/h"
  tail -n +2 "$o" | cut -d, -f1,2,3 | LC_ALL=C sort -t, -k1,1 > "$t/o"
  LC_ALL=C join -t, -a1 -e NA -o auto "$t/pse2" "$t/h" > "$t/pseh"              # 3: LEFT JOIN hosts on ip
  LC_ALL=C join -t, -a1 -e NA -o auto "$t/pseh" "$t/o" | cut -d, -f2- > "$t/m"   # 4: LEFT JOIN os on ip
  { echo "ip,port,proto,state,service,version,enum_source,enum_finding,hostname,mac,os_guess,os_family"
    LC_ALL=C sort -t, -k1,1V -k2,2n "$t/m"; } > "$out"
  rm -rf "$t"
}

# sl_export SRC DIR PREFIX [VERSION] : noclobber write + checksum line
sl_export() {
  local src="${1:?src}" dir="${2:-out}" pre="${3:-scan}" ver="${4:-1}" ts name
  [[ -r $src ]] || { _err "missing file: $src"; return 2; }
  mkdir -p "$dir"; ts="$(date +%Y%m%d-%H%M%S)"
  name="$dir/${pre}_${ts}_v${ver}.csv"
  ( set -o noclobber; cat "$src" > "$name" ) 2>/dev/null || { _err "refusing to overwrite $name"; return 1; }
  ( cd "$dir" && sha256sum "$(basename "$name")" >> checksums.sha256 )
  echo "$name"
}

# sl_session_diff OLD NEW OUTDIR : compare two scan sessions on key ip:port:proto
# writes retained.csv updated.csv gone.csv new.csv ; prints counts
sl_session_diff() {
  local a="${1:?old}" b="${2:?new}" d="${3:-.}" f
  for f in "$a" "$b"; do [[ -r $f ]] || { _err "missing file: $f"; return 2; }; done
  mkdir -p "$d"
  tail -n +2 "$a" | awk -F, -v OFS=, '{print $1":"$2":"$3,$4}' | LC_ALL=C sort -t, -k1,1 > "$d/old.k"
  tail -n +2 "$b" | awk -F, -v OFS=, '{print $1":"$2":"$3,$4}' | LC_ALL=C sort -t, -k1,1 > "$d/new.k"
  LC_ALL=C join -t, "$d/old.k" "$d/new.k" | awk -F, '$2==$3' > "$d/retained.csv"
  LC_ALL=C join -t, "$d/old.k" "$d/new.k" | awk -F, '$2!=$3' > "$d/updated.csv"
  LC_ALL=C join -t, -v1 "$d/old.k" "$d/new.k" > "$d/gone.csv"
  LC_ALL=C join -t, -v2 "$d/old.k" "$d/new.k" > "$d/new.csv"
  echo "retained=$(wc -l < "$d/retained.csv") updated=$(wc -l < "$d/updated.csv") unmatched_old=$(wc -l < "$d/gone.csv")
      new=$(wc -l < "$d/new.csv")"
}
```

Appendix B - scan_pipeline.sh (integrated driver)

```
#!/usr/bin/env bash
# scan_pipeline.sh - integrated pipeline: import -> validate -> clean -> merge -> export
set -euo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/scanlib.sh"
trap 'echo "scan_pipeline: error near line $LINENO" >&2' ERR
trap '[[ -n "${WORK:-}" ]] && rm -rf "$WORK"' EXIT
cnt() { echo $(( $(wc -l < "$1") - 1 )); }

# run_pipeline [SUBNET] [SCANTYPE] [DATADIR] [OUTDIR]
run_pipeline() {
  local subnet="${1:-192.168.56}" stype="${2:-tcpsyn}" dir="${3:-data7}" out="${4:-out7}"
  local n f; WORK="$(mktemp -d)"; SCAN_SUBNET="$subnet"; mkdir -p "$out"
  for n in hosts ports services os enum; do
    [[ -r $dir/$n.csv ]] || { echo "scan_pipeline: missing $dir/$n.csv" >&2; return 2; }
    sl_check_header "$dir/$n.csv" "schema/$n.hdr" || return $?
  done
  : > "$out/review.csv"
  for n in hosts ports services os enum; do
    sl_normalize "$dir/$n.csv" "$WORK/$n.1"
    sl_validate  "$WORK/$n.1" "$WORK/$n.2" "$out/rejected_$n.csv" "$subnet"
    printf '%-9s input=%-3s rejected=%-3s' "$n" "$(cnt "$dir/$n.csv")" "$(cnt "$out/rejected_$n.csv")"
    case $n in
      ports|services) sl_fill_missing "$WORK/$n.2" "$WORK/$n.3" version unknown
                      sl_dedupe "$WORK/$n.3" "$WORK/$n.4" "$out/review.csv" ;;
      *) cp "$WORK/$n.2" "$WORK/$n.4" ;;
    esac
    [[ $n == services ]] && { sl_std_service "$WORK/services.4" "$WORK/services.5" "$out/review.csv" "$SCAN_LOOKUP"; cp
        "$WORK/services.5" "$WORK/services.4"; }
    echo " cleaned=$(cnt "$WORK/$n.4")"
  done
  sl_merge_master "$WORK/ports.4" "$WORK/services.4" "$WORK/hosts.4" "$WORK/os.4" "$WORK/enum.4" "$WORK/master.csv"
  echo "merged rows=$(cnt "$WORK/master.csv") (base ports rows=$(cnt "$WORK/ports.4"))"
  cp "$WORK/master.csv" "$out/master_latest.csv"
  f="$(sl_export "$WORK/master.csv" "$out" "${subnet}_${stype}" 1)"
  echo "exported=$(basename "$f") final rows=$(cnt "$f") sha256=$(sha256sum "$out/master_latest.csv" | cut -c1-16)"
}
if [[ "${BASH_SOURCE[0]}" == "$0" ]]; then run_pipeline "$@"; fi
```

Appendix C - svc_lookup.csv (service-name lookup file)

```
variant,standard
www,http
web,http
http,http
http-alt,http
http-proxy,http
https,https
ssl/http,https
https-alt,https
ftp,ftp
ssh,ssh
openssh,ssh
telnet,telnet
smtp,smtp
mysql,mysql
mysql-db,mysql
msrpc,msrpc
netbios-ssn,netbios-ssn
netbios,netbios-ssn
microsoft-ds,microsoft-ds
ms-ds,microsoft-ds
smb,microsoft-ds
ms-wbt-server,ms-wbt-server
rdp,ms-wbt-server
adb,adb
rmiregistry,rmiregistry
rmi,rmiregistry
ncp,ncp
tcpmux,tcpmux
jetdirect,jetdirect
snmp,snmp
tftp,tftp
```